



Python Cheatsheet Final

Introduction to python (Universiteit van Amsterdam)



Scan to open on Studocu

Method on Immutable

s1 = 'Uva Amsterdam'
s1.upper() is wrong
s1 = s1.upper() is corr
string cant be changed
but new string is okay

List II

l1.sort()
→ sorts list itself
l1.index('a')
→ finds index of
first occurrence of a

Dictionaries

Creating new dictionary:
capitals = {}
capitals = dict()
capitals = {'country': 'capital'}
del(capitals['Belgium'])

Adding Values

capitals['Neth'] = 'The Hague'
capitals.update({'Neth': 'Amsti', ...})
Change key:value
code above
capitals['Neth'] = 'Amsti'

d.keys() seq of list
d.values() seq of list
d.items() seq of tuples
Check Existence
if 'key' in 'dictionary'

Zip Function:

Goes through items and
adds items into tuple by
index order, for dicts it
takes the keys auto
Can zip more than 2 seqs

Add Elements to List

l1 += ['pookie', 'bear']
l1.append('pookie')
(only one item possible)
l1.remove('ex pookie')

break: skip to the first statement after the loop
continue: go back to the statement with the for keyword

Boolean Evaluations

T or T = T	T and T = T
T or F = T	T and F = F
F or F = F	F and F = F
not True = F	not False = T

Hierarchy:

brackets
not
and
or (left to right)

Tuples

Create tuple: t = (1, 2, 3) or t = 1, 2, 3 or t = (1,
t(1) is an integer, property of tuples: x, y = y, x

Args and Kwargs

Args: name args attached to list that contains all arguments, args is a convention
def adder(n1, n2, *args) here python first fills w/ defined then undefs
Kwargs:

```
def calc(**kwargs):  
    return kwargs['first'] + 2 * kwargs['second'] + \  
        3 * kwargs['third'] + 4 * kwargs['fourth'] + \  
        5 * kwargs['fifth'] + 6 * kwargs['sixt']  
  
print(calc(third=4, fourth=1, second=2, first=3, fifth=2, sixth=5))  
# Onscreen: 63
```

Make sure to follow right order or errors can occur:

```
def calc(first, second, *args, **kwargs):  
    return first + second * 2 + args[0] * 3 + args[1] * 4 + \  
        5 * kwargs['fifth'] + 6 * kwargs['sixt']  
  
print(calc(3, 2, 4, 1, sixth=5, fifth=2)) # Onscreen: 63
```

(Global Local) Functions

Returns None if no return
global name: defined outside any function
known everywhere, also inside functions,
if defined inside function, local > global

```
def main(a):  
    b = b  
    return a  
b = 1  
print(main(1))  
# Onscreen: UnboundLocalError:
```

```
def main(a):  
    global b  
    b = b  
    return a  
b = 1  
print(main(1)) # 1
```

```
def changer(n1):  
    n1 = n1 + 1  
    return n1  
a = 1  
a = changer(a)  
print(a) #2
```

```
def changer():  
    global n1  
    n1 = n1 + 1  
    n1 = 1  
    changer()  
    print(n1) #2
```

```
def changer(n1):  
    n1 = 2  
    return n1  
n1 = 1  
changer(n1)  
print(n1) # Onscreen: 1
```

l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]]
df1 = pd.DataFrame(l1)
The result of this code will be:

	0	1	2
0	A	153	55
1	B	160	70
2	C	150	60

• import pandas as pd
l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]]
df1 = pd.DataFrame(l1)

• The result of this code will be:

	0	1	2
0	A	153	55
1	B	160	70
2	C	150	60

Creating df with list of lists makes the index and columns indecies, data goes row by row

l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]]

Create Series – name is opt. if no index rows will start at 0 and continue//+cant index series

```
First      153 d1 = {'First': 153, 'Second': 160, 'Third': 150}  
Second     160 s1 = pd.Series(d1, name = 'Length')  
Third      150 l1 = [153, 160, 150]  
Name: Length s1 = pd.Series(l1, name = 'Length', index=['First', 'Second', 'Third'])
```

Sort by labels: df1=df1.sort_index(axis=1, ascending=False, default 1 (cols)&True
Row values: df1=df1.sort_values(by='First', ascending=False, axis=1)

loc: df1.loc[row_selection, col_selection], select w slicing (start:end(inclusive))
or list. Can leave out col_selection, default is all.

iloc: similar but rel. position instnd of names, end is not inclusive, -ve slicing possible

df1.loc[[2],13]/df1.iloc[[-3],-3] vs df1.loc[2,13]/ df1.iloc[-3,1] is bracks = series,

s1.name for df1.loc[2] gives name or row/col dep on whats called no brack = int

To change data of rows w/names 3 and 0

df1.loc[[3,0]] = [['E', 100, 40, 20], ['F', 101, 41, 21]]

Can also use Booleans to select rows/cols with .loc

```
df1=df1.sort_values(by='First', ascending=False, axis=1)  
      Length  Name  Weight  
first      A    153     55  
second     B    160     70  
third      C    150     60
```

Use of correct quotations

Escape characters:

```
print("it\"s's") # Onscreen: it"s's
print('it\"s\'s') # Onscreen: it"s's
```

Triple Quotes:

```
print("""it"s's""") # Onscreen: it"s's
print('\'it"s's\'') # Onscreen: it"s's
```

```
\\=\\
\\'= '
\\\"= \"
\\n= go to the next line
\\t= insert a tab
```

```
print('The info, it\'s on the next\\following line in two
columns:\\n\\t2')
# Onscreen: The info, it's on the next\\following line in two
columns:
# 1 2
```

String Methods/Manipulations

Sorted list: `print(sorted('acbd'))`

Onscreen: ['a', 'b', 'c', 'd']

`len('abcd') >> 4`

`'abcd'.index('b') >> 1`

remember: strings immutable thus:

- string methods can only ret new

obj method w/o return returns **None**

```
s1 = 'abcd'
s1 = s1.capitalize()
print(s1)
>> capitalizes s1
s1 = 'abcd'
s1.capitalize()
print(s1)
>> prints abcd
```

```
print('ABcd'.upper()) # Onscreen: ABCD .upper()
print('ABcd'.lower()) # Onscreen: abcd .lower()
print('ABCD'.isupper()) # Onscreen: True .isupper()/islower()
print('abcd'.islower()) # Onscreen: True
print('abcd'.capitalize()) # Onscreen: Abcd
Capitalize, lower, upper, islower, isupper
```

Split

`string.split('argument', nr_of_times)`

No specified: splits with spaces, \n and \t

Removes what you are splitting with

If argument not in str returns same list

`string.rsplit(...)` starts from right

```
s1 = 'axbcxdefxg h\nij\tk'
print(s1.split('x', 2))
```

```
['a', 'bc', 'defxg h\nij\tk']
```

```
s1 = 'axbcx\ndefxg \nhij'
print(s1.split('\n'))
print(s1.splitlines())
['axbcx\ndefxg ', 'hij']
['axbcx', 'defxg ', 'hij']
```

Join

`join_with.join(list/argument)`

```
l1 = ['abc', 'de', 'fg', 'h']
print('xy'.join(l1))
>> abcxydexyfgxyh
print(''.join(l1))
>> abcdefgh
```

Sort strings:

```
a = sorted('0123456789', key=lambda x: str(int(x) % 2 == 1) + x))
['00', '02', '04', '06', '08', '11', '13', '15', '17', '19']
print(''.join(a)) '0246813579'
```

Sorted example explanation:

`int(x) % 2 == 1`: expression checks if the integer value of x is odd
True for odd digits, False for even thus the 00, 02,... are the keys
Then the sorted function sorts the characters based off of the keys
0 has key 00, 2 has 02 and so on

note: `l1 = s1.sort()` is wrong, makes l1 None
use `l1 = sorted(s1)`

Strip :

`string.strip(argument)` – strips on both sides and stops when it doesn't find argument anymore

>> The default is removing spaces, new lines (\n), and tabs (\t)

`string.lstrip(argument)` – only right

`string.rstrip(argument)` – only left

```
s1 = ' fgedee '
#spaces after # are intentional
print(s1.strip(' e')) #fged
print(s1.strip()) #fgedee
print(s1.lstrip(' e')) #fgedee
print(s1.lstrip()) #fgedee
print(s1.rstrip(' e')) # fged
print(s1.rstrip()) # fgedee
```

Find and Index

Both return index for what you are looking for

`string.find('e', start, stop)` Only different is when cant find, find returns value of -1

`string.index('e')` Index will throw an error

Replace

`string.replace('old', 'new')`

F – Strings

`f"{}value:formatting}{...}"`

If you leave out formatting Python will apply string formatting

```
s1 = 'tekst'
t1 = (1, 2, 3)
l1 = [4, 5, 6]
capitals = {'Netherlands' : 'Amsterdam'}
print(f'{s1} {s1.upper()} {t1[2]} {l1} {capitals["Netherlands"]}')
# Onscreen: tekst TEKST 3 [4, 5, 6] Amsterdam
a = 10
b = 'Amsterdam'
print(f'{a}={b}') # Onscreen: a=10 b='Amsterdam'
```

Dates and Times

`datetime.date` year, month and day

`datetime.time` hour, min, sec, microsec, tzinfo

`datetime.datetime` has info of both above

`datetime.datetime` positional parameters: year>month>day>hr>min>sec

`timedelta` only has days, seconds, and microseconds

```
d = datetime.now()
print(f'{d.year = }, {d.month = }, {d.day = }, {d.hour = }, {d.minute = }, {d.second = }, {d.microsecond = }')
# gives: d.year = 2024, d.month = 5, d.day = 28, d.hour = 11, d.minute = 9, d.second = 54, d.microsecond = 538983
```

```
from datetime import datetime
birthday = datetime(1923, 10, 26)
today = datetime.now()
age = today - birthday
print(f'{age.days}{age.seconds}') # On screen: 36732 14249
print(type(age)) # On screen: Timedelta
```

strftime example

```
d = datetime.now()
print(f'')
(d.strftime('Year with century: %Y') = 1
```

2024-05-28 11:29:38.498978

```
d.strftime('Year with century: %Y') = 'Year with century: 2024'
d.strftime('Year without century: %y') = 'Year without century: 24'
d.strftime('Full month name: %B') = 'Full month name: May'
d.strftime('Month number: %m') = 'Month number: 05'
d.strftime('Day of the month: %d') = 'Day of the month: 28'
d.strftime('Full weekday name: %A') = 'Full weekday name: Tuesday'
d.strftime('Weekday number: %a') = 'Weekday number: 2'
d.strftime('Week number of the year: %U') = 'Week number of the year: 21'
d.strftime('Day number of the year: %j') = 'Day number of the year: 149'
d.strftime('Hour number: %H') = 'Hour number: 11'
d.strftime('Minute number: %M') = 'Minute number: 29'
d.strftime('Second number: %S') = 'Second number: 38'
```

```
from datetime import datetime, timedelta
today = datetime.now()
future_day = datetime.now() + timedelta(days=100)
future_day = datetime.now() + timedelta(weeks=14, days=2)
future_day = datetime.now() + timedelta(hours=100*24)
future_day = datetime.now() + timedelta(minutes=100*24*60)
future_day = datetime.now() + timedelta(seconds=100*24*60*60)
print(future_day.strftime('%Y-%m-%d')) #Output: 2024-09-05
```

Classes

Everything indented past class is part of class definition

stops when code is on same indentation as class

When class called on Python creates new object and runs it

It through the `__init__` with name of var as the self

>> Creation is called instantiation, convention to capitalize class

`__init__` can only have None as return statement

```
class Rectangle:
    objects_in_existence = 0
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
        Rectangle.objects_in_existence += 1
    def size(self):
        return self.length * self.width
    def info(self):
        return f'This rectangle is {self.length} by {self.width}'
    def __del__(self):
        Rectangle.objects_in_existence -= 1
rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3)
print(rectangle_1.size()) # Onscreen: 6
print(rectangle_2.info()) # Onscreen: This rectangle is 3 by 1
print(Rectangle.objects_in_existence) # Onscreen: 2
print(rectangle_1.length, rectangle_1.width) #2 3
```

Strptime

```
d = datetime.strptime('2024$05%20', '%Y$%m$%d')
d = datetime.strptime('24$05%20', '%y$%m$%d')
d = datetime.strptime('24$05%20', '%y$%m$%d')
d = datetime.strptime('2024*141', '%Y*%j')
d = datetime.strptime('2024*20xyz1', '%Y*%uxyz%w')
d = datetime.strptime('202420Monday', '%Y%u%A')
d = datetime.strptime('00:00:00 20-05-2024', '%H:%M:%S %d-%m-%Y')
print(d) #Output (all): 2024-05-20 00:00:00
```

```
print(sorted('0123456789', key=lambda x: str(int(x) % 2 == 1) + x)) print(sorted('abcdeFGHIJKLmNopQRStUVwXyZ', key=lambda x: x.upper()))
['0', '2', '4', '6', '8', '1', '3', '5', '7', '9'] ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N',
```

```
print(sorted('abcdeFGHIJKLmNopQRStUVwXyZ', key=lambda x: str(x.lower() not in 'aeiou') + x.lower())) print(sorted('aBCdEFGHIJKLmNopQRStUVwXyZ'))
['a', 'E', 'I', 'O', 'U', 'B', 'C', 'D', 'F', 'G', 'H', 'J', 'K', 'L', 'M', 'N', 'P', 'Q', 'R', 'S', 'T', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```